

## Passing parameters to the functions: [parameter passing techniques]

In C-Language, we can send the arguments to the functions in 2 ways.

1. Call by value / pass by value.
2. Call by address / pass by address. [ call by reference ]

### Call by value / pass by value:

In call by value we are sending actual parameter value to the formal parameter. Later there is no relation is maintained in between actual and formal parameters. Due to this any change in formal parameter doesn't effects the value of actual parameter.

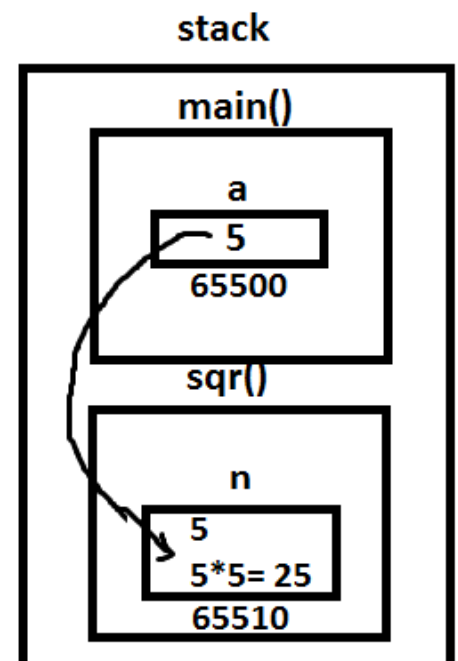
Eg: 1

```
#include<stdio.h>
#include<conio.h>

void sqr(int n)
{
    n = n * n;
} /* n deleted after the function execution */

void main()
{
    int a=5;
```

### CALL BY VALUE



```
clrscr();  
printf("Before function call a = %d\n",a);  
sqr(a); /* fun calling */  
printf("After function call a = %d", a):  
getch();  
}
```

### **Output:**

Before function call a = 5

After function call a = 5

### **Eg: 2 swapping of two integers**

```
#include<stdio.h>  
#include<conio.h>  
void swap(int a, int b)  
{  
    int temp=a;  
    a=b;  
    b=temp;  
}  
void main()
```

```

{
int a=5, b=7;
clrscr();
printf("Before fun call a=%d, b=%d\n" , a , b);
swap(a, b);
printf("After fun call a=%d, b=%d", a , b);
getch();
}

```

### **Output:**

Before fun call a=5, b=7

After fun call a=5, b=7

The screenshot shows the Turbo C++ IDE with the following code in the editor:

```

Line 12 Col 6 Insert Indent Tab Fill Unindent * E:NONAME.C
#include<stdio.h>
#include<conio.h>
void show(int x) /* fun def, x is formal par */
{
x=200;
} /* x deleted */
void main()
{
int x=100; /* local var */
clrscr();
printf("Before fun call x=%d\n",x);
show(x); /* fun calling , x is actual par */
printf("After fun call x=%d",x);
getch();
}

```

The IDE interface includes a menu bar (File, Edit, Run, Compile, Project, Options, Debug, Break/watch), a toolbar, and a status bar at the bottom showing system information (6:44 PM, 2/13/2023) and a Windows activation notice.

```
Before fun call x=100
After fun call x=100
```

TC

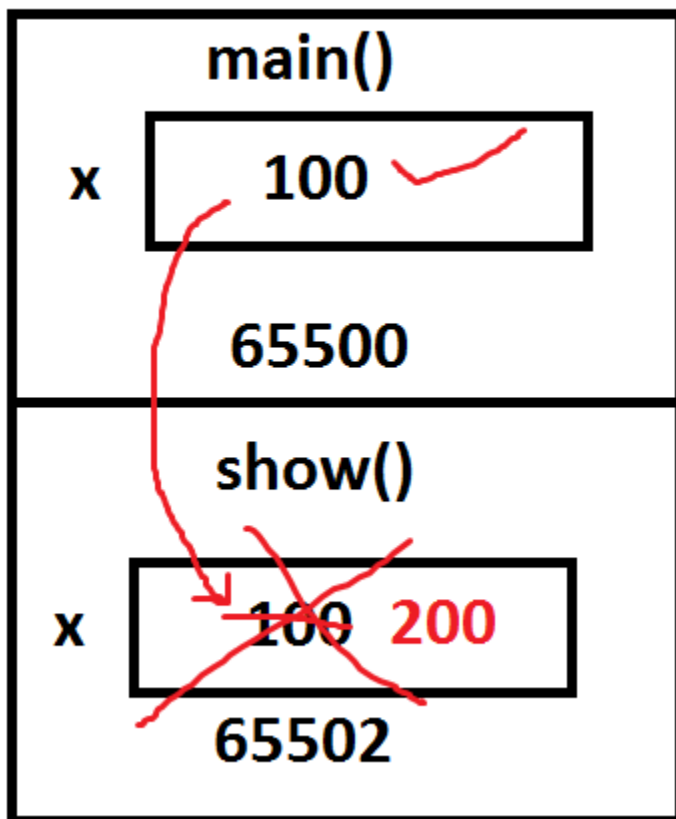
You are screen sharing Stop Share

Mute Start Video Security Participants Chat New Share Pause Share Annotate Apps More

Activate Windows  
Go to PC settings to activate Windows.

6:44 PM  
2/13/2023

## Call by /pass By value



## Call by address [Reference]:

In call by address, the address of actual parameter is passed to formal parameter. Due to this the formal parameter should be declared as a pointer. Then only the formal parameter receives the actual parameter address. Due to this any changes in formal parameter effects in actual parameter address i.e. actual parameter value.

Hence pointers allows the local variables to access outside the functions and this process is called call by address / reference.

It is very much useful in handling the strings, arrays etc outside the functions.

Eg: 1

```
#include<stdio.h>
#include<conio.h>
```

```
void sqr(int *n)
```

```
{
```

```
*n = *n * *n;
```

```
}
```

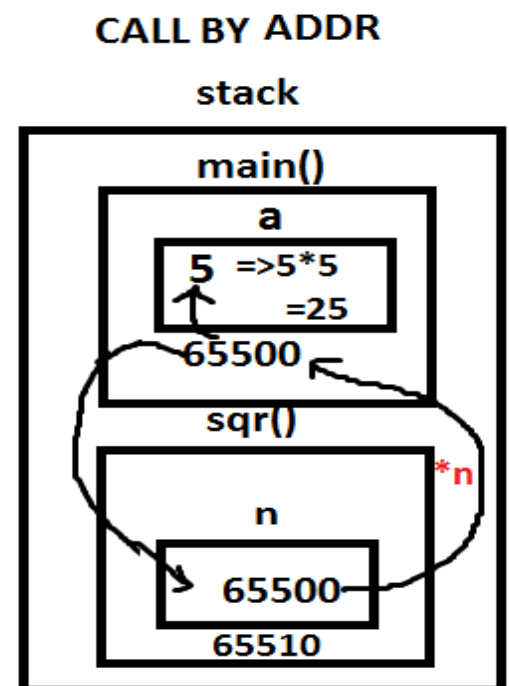
```
void main()
```

```
{
```

```
int a=5;
```

```
clrscr();
```

```
printf("Before function call a = %d\n " ,
```



```
a);  
sqr(&a); /* fun calling with address */  
printf("After function call a = %d " , a);  
getch();  
}
```

### **Output:**

Before function call a = 5

After function call a = 25

### **Eg: 2 Swap of two integers**

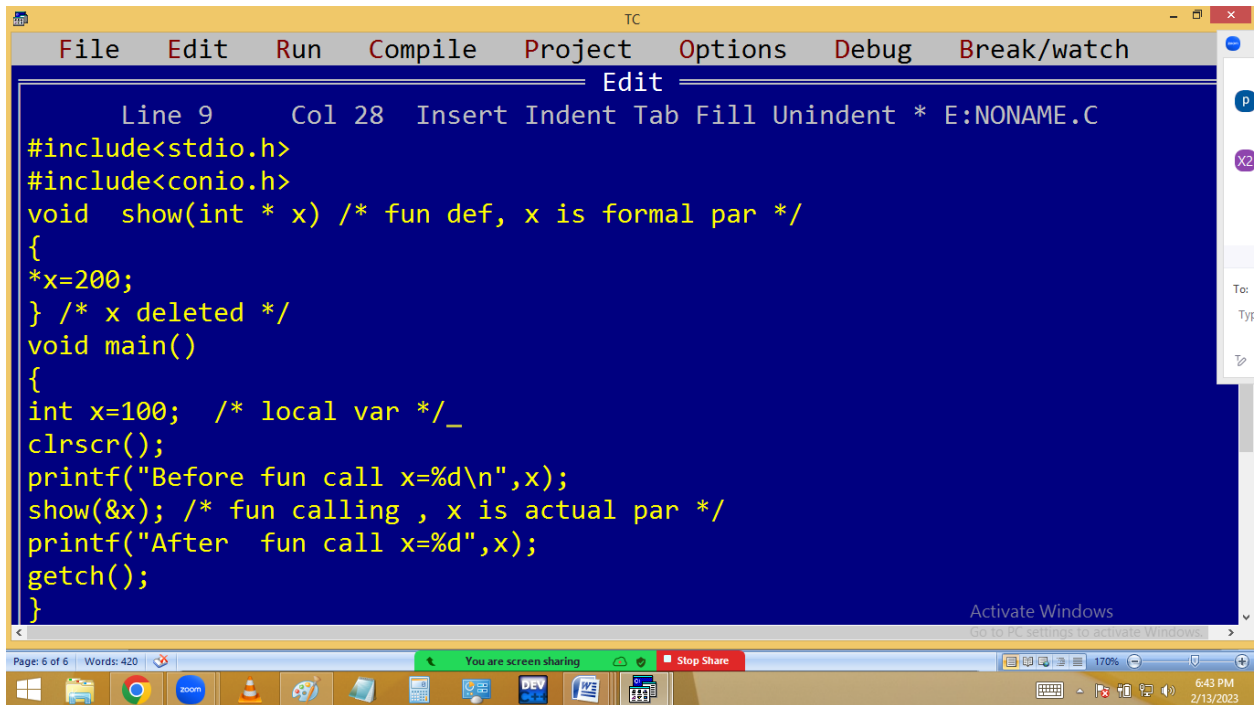
```
#include<stdio.h>  
  
#include<conio.h>  
  
void swap(int *a, int *b)  
{  
    int temp=*a; *a = *b; *b=temp;  
}  
  
void main()  
{  
    int a=5, b=7;  
    clrscr();
```

```
printf("Before fun call a=%d, b=%d\n", a ,b);  
swap(&a, &b);  
printf("After fun call a=%d, b=%d", a ,b);  
getch();  
}
```

### **Output:**

Before function call a=5, b=7

After function call a=7, b=5



```
TC  
File Edit Run Compile Project Options Debug Break/watch  
Edit  
Line 9 Col 28 Insert Indent Tab Fill Unindent * E:NONAME.C  
#include<stdio.h>  
#include<conio.h>  
void show(int * x) /* fun def, x is formal par */  
{  
  *x=200;  
} /* x deleted */  
void main()  
{  
  int x=100; /* local var */_  
  clrscr();  
  printf("Before fun call x=%d\n",x);  
  show(&x); /* fun calling , x is actual par */  
  printf("After fun call x=%d",x);  
  getch();  
}
```

Page: 6 of 6 Words: 420 You are screen sharing Stop Share 170% 6:43 PM 2/13/2023

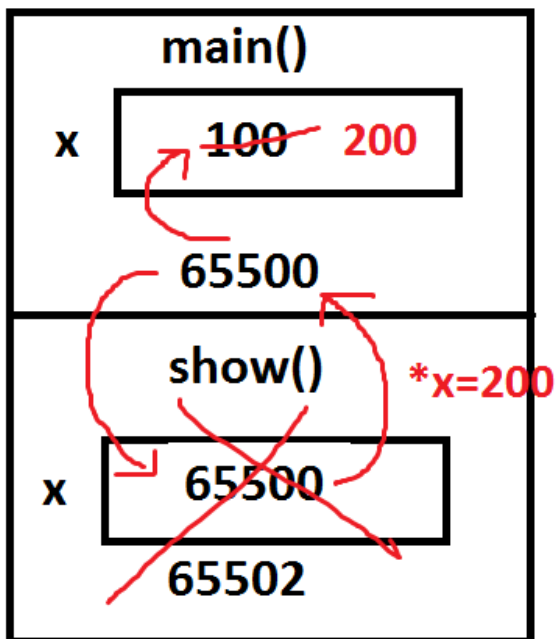
```
TC
Before fun call x=100
After fun call x=200
```

Activate Windows  
Go to PC settings to activate Windows.

You are screen sharing Stop Share

6:43 PM  
2/13/2023

call / pass by address



**\*x = 200**  
**x value is 65500**  
**\* means value at 65500 = 200**



## Passing array / string to function?

String/array is implicit pointer i.e. string / array variable stores base address. Due to this when string/array is passed to a function, implicitly base address is passed and formal parameter becomes pointer and it receives this address. Hence any change occurred in formal parameter, effects on actual parameter value also.

We can declare string / array formal parameter in 3 ways.

1. With size eg: char st[50] / int a[3]
2. Without size eg: char st[ ] / int a[ ]
3. As a pointer eg: char \* st / int \*a

We can pass string / array actual parameter with or without address.

**Eg:**

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void reverse( char st[10] ) or st[ ] or *st
```

```
{
```

```
strrev(st);
```

```
}
```

```
void main()
```

```
{
```

```
char s[10]="abcd";
```

```
clrscr();
```

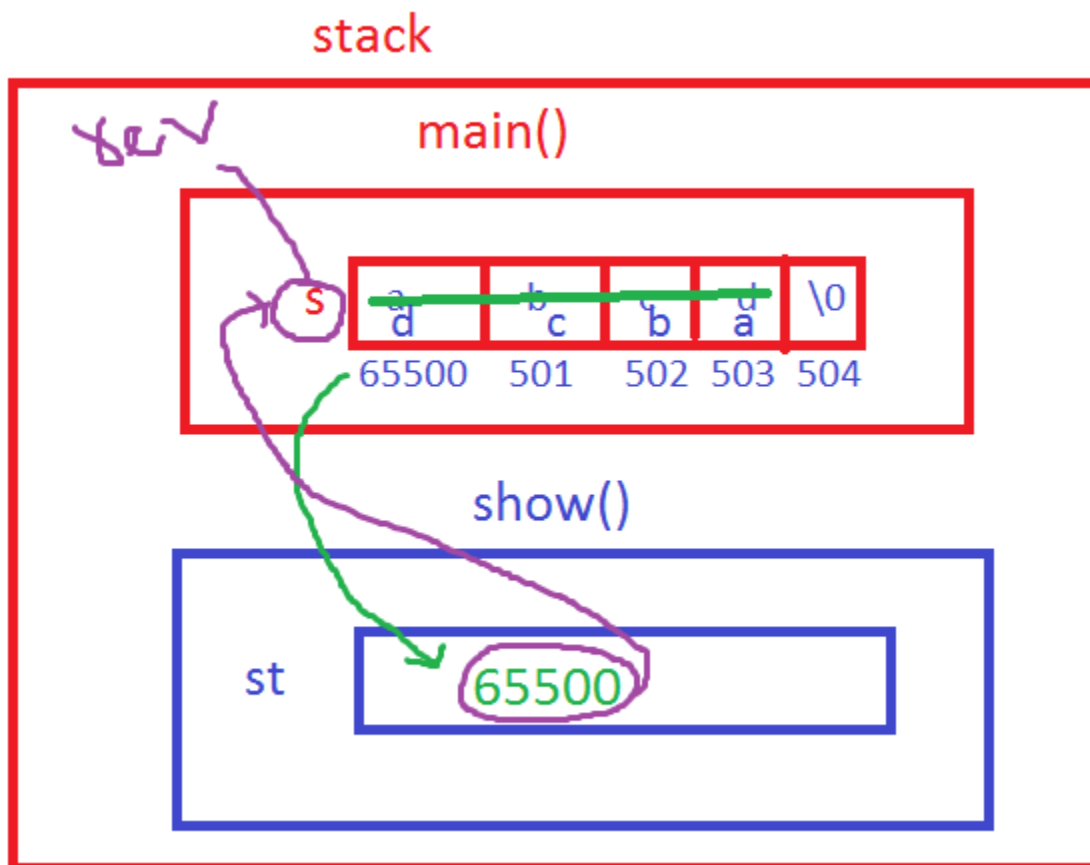
```
reverse(s); or reverse(&s);
```

```
printf("String = %s", s);
```

```
getch();
```

```
}
```

**O/P: String = dcba**



### Passing array to function:

```
#include<stdio.h>
#include<conio.h>
void show(int a[3]) or a[ ] or *a
{
a[0]=100; a[1]=200; a[2]=300;
}
void main()
{
int a[3]={10,20,30};
clrscr();
show(a); or show(&a);
printf("Array elements %d %d %d",a[0],a[1],a[2]);
getch();
}
```

**O/P: Array elements 100 200 300**

### **Passing two – dimensional array to function.**

```
#include<stdio.h>
#include<conio.h>
void show( int (*a)[3] ) or a[2][3] or a[ ][3]
```

```

{
a[0][0]=10; a[1][2]=60;
}

void main()
{
int a[2][3]={1,2,3,4,5,6};
show(a); /* fun calling */
printf("a[0][0]=%d, a[1][2]=%d",a[0][0],a[1][2]);
getch();
}

```

**Output: a[0][0]=10, a[1][2]=60;**

```

#include<stdio.h>

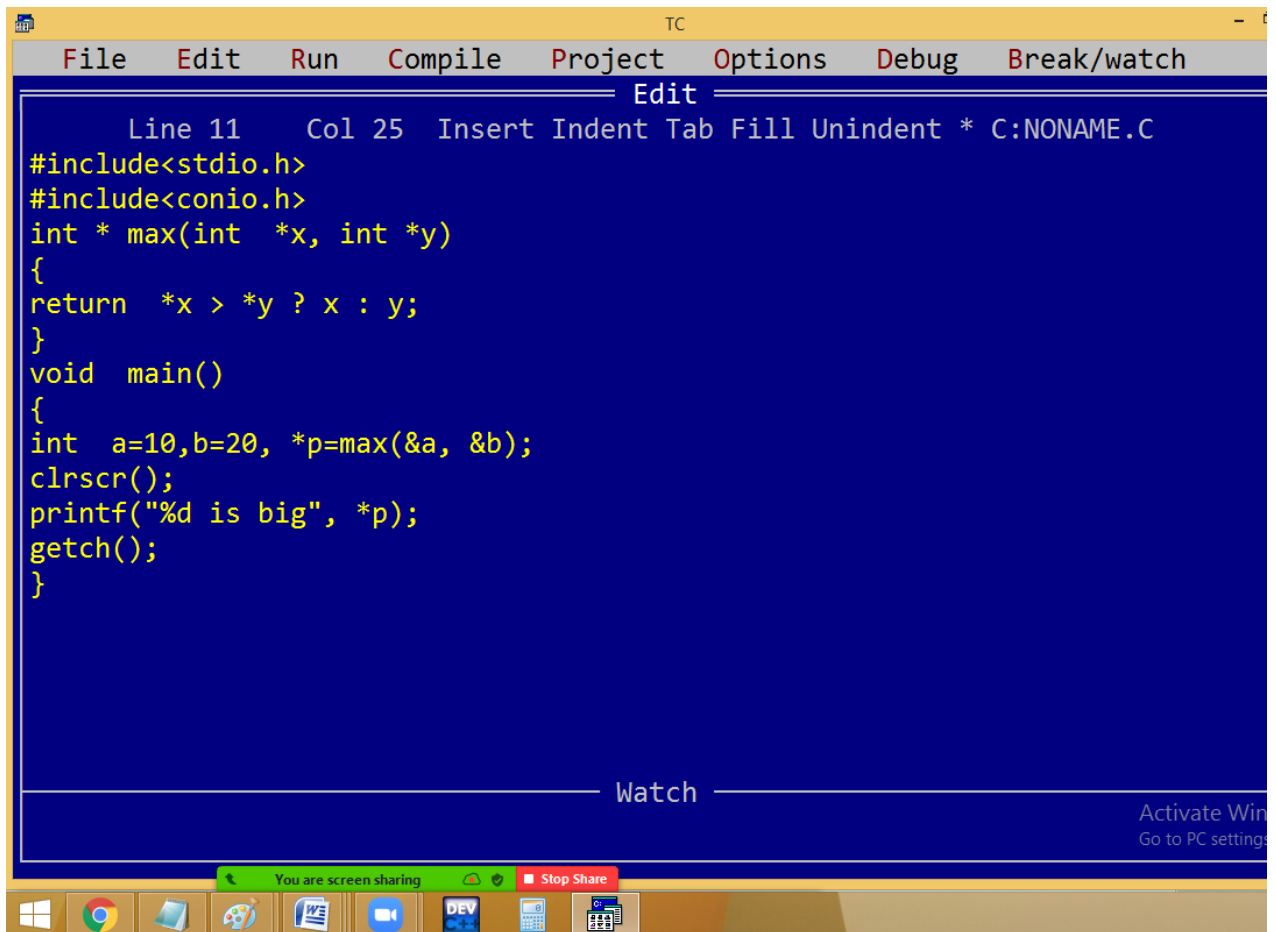
#include<conio.h>

void show(int (*a)[3]) /* or a[2][3] or a[][3]*/
{
int r,c;
printf("Elements are\n");
for(r=0;r<2;r++)

```

```
{
for(c=0;c<3;c++)
{
printf("%4d",*(*(a+r)+c)); /*or a[r][c]*/
}
printf("\n");
}
}
void main()
{
int a[2][3]={1,2,3,4,5,6};
clrscr();
show(a); /* fun calling */
getch();
}
```

Function returning address [ pointer ]



The screenshot shows the Turbo C++ IDE with a menu bar (File, Edit, Run, Compile, Project, Options, Debug, Break/watch) and a toolbar. The main window displays a C program in a dark blue editor. The code defines a function `max` that returns the larger of two integers and a `main` function that uses it with values 10 and 20. The output window at the bottom shows the result "20 is big".

```
Line 11 Col 25 Insert Indent Tab Fill Unindent * C:NONAME.C
#include<stdio.h>
#include<conio.h>
int * max(int *x, int *y)
{
return *x > *y ? x : y;
}
void main()
{
int a=10,b=20, *p=max(&a, &b);
clrscr();
printf("%d is big", *p);
getch();
}
```

Watch

Activate Win  
Go to PC settings

You are screen sharing Stop Share

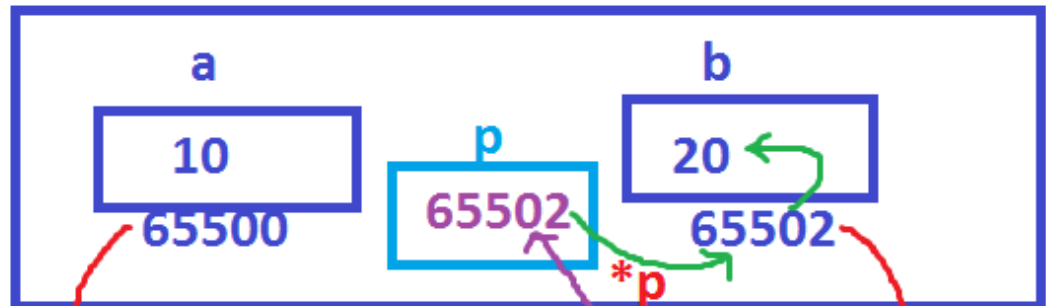


The screenshot shows the output window of the Turbo C++ IDE, which is a black rectangle with a yellow border. It displays the text "20 is big" in a white, monospaced font.

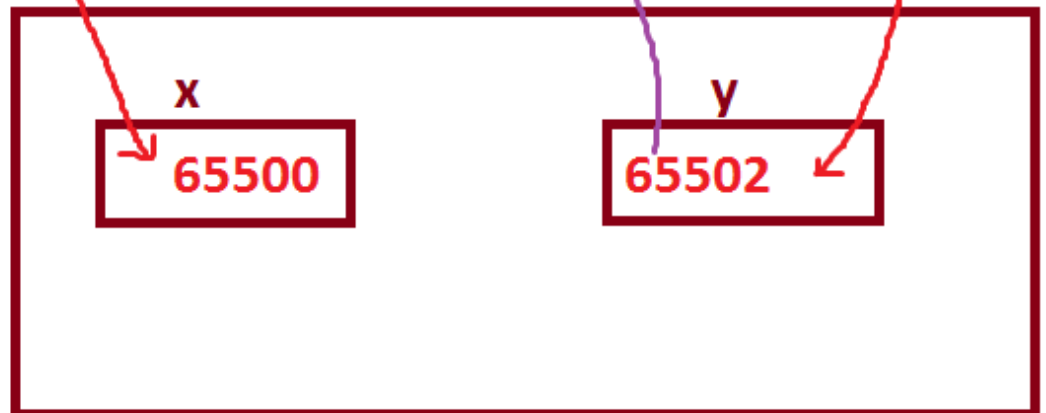
20 is big

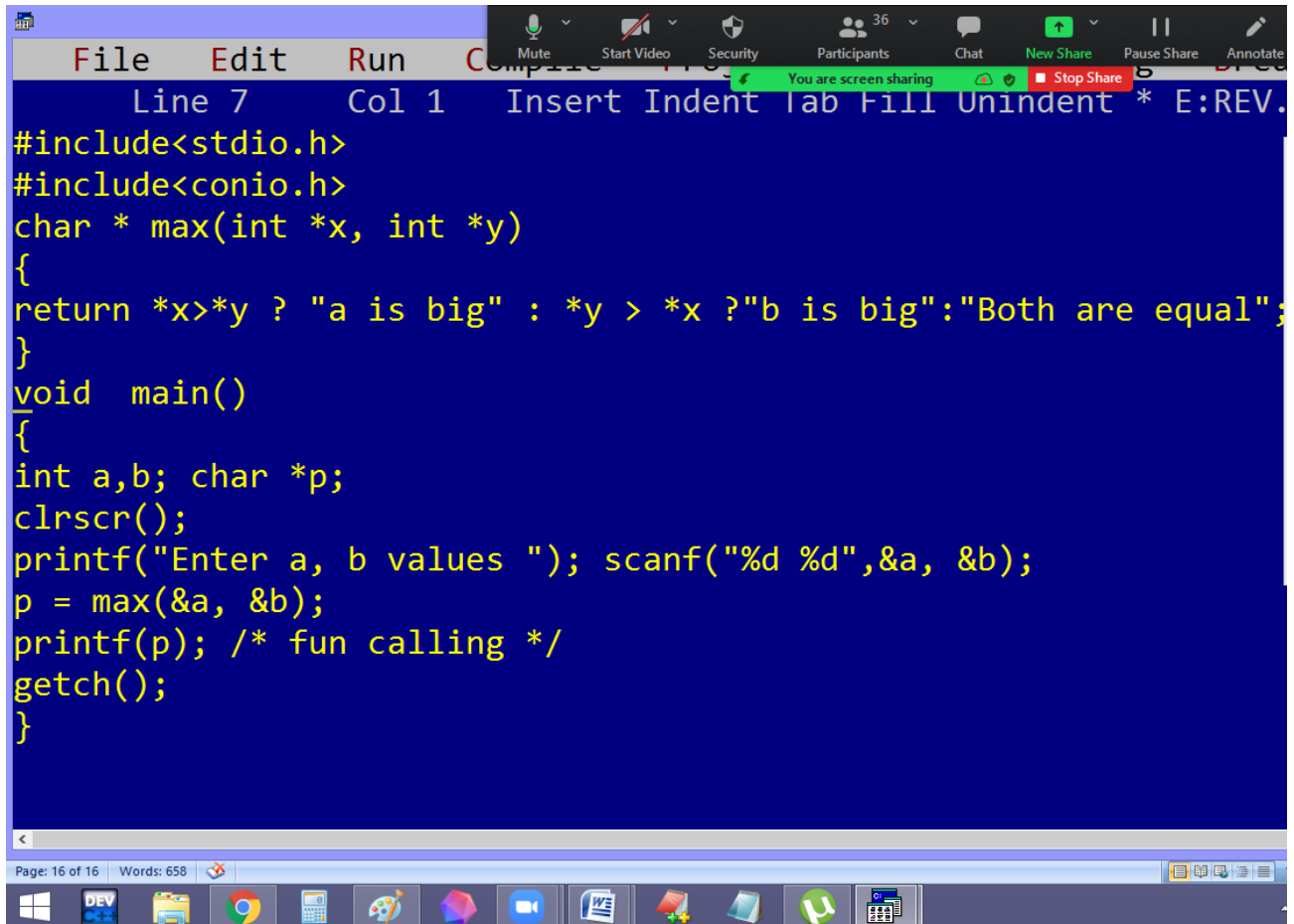
stack

main()



max()



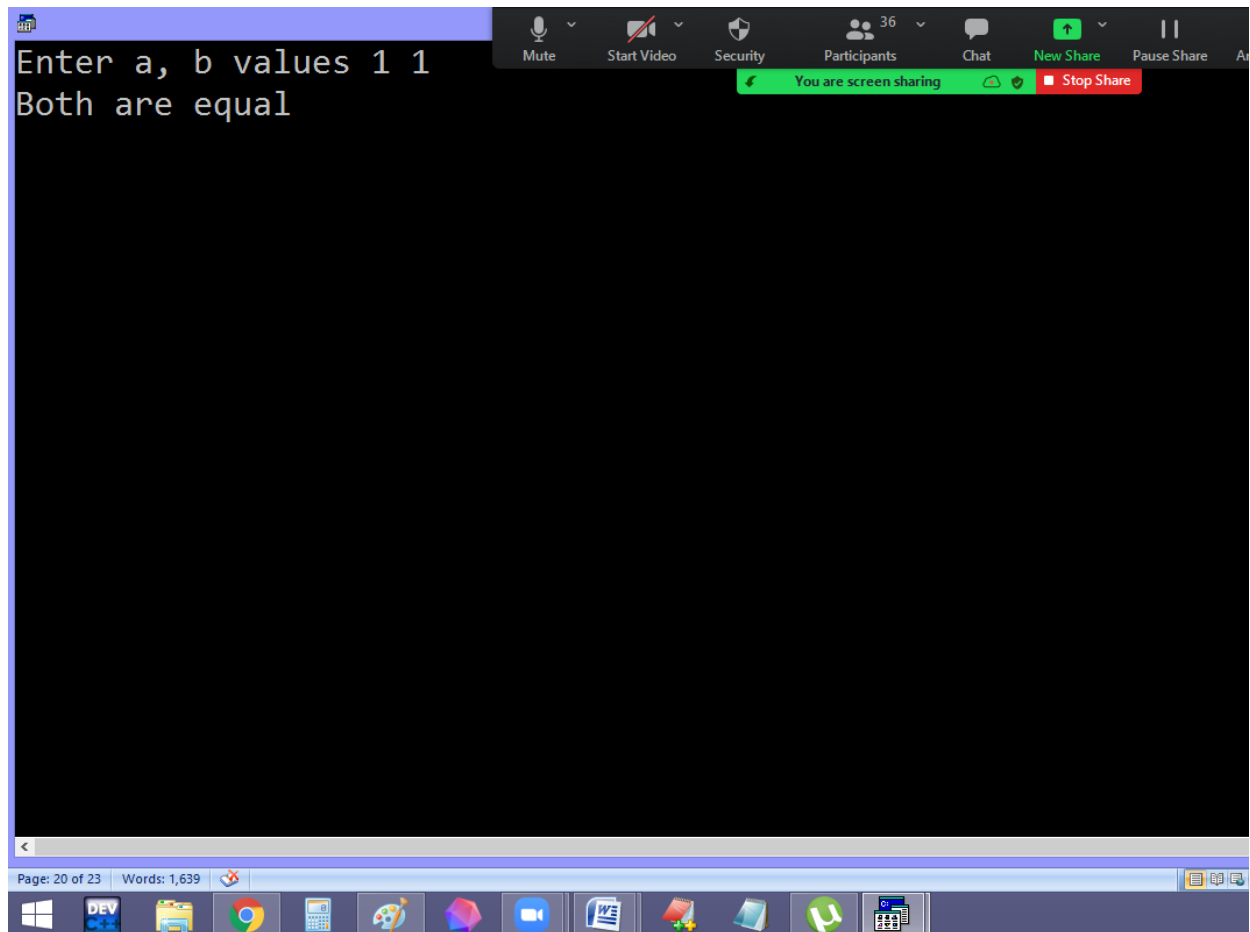


The image shows a screenshot of a code editor window with a dark blue background and yellow text. The code is a C program that defines a function `max` to compare two integers and returns a string indicating which is larger or if they are equal. The `main` function prompts the user to enter values for `a` and `b`, calls the `max` function, and prints the result. A screen sharing overlay is visible at the top, showing a toolbar with options like Mute, Start Video, Security, Participants, Chat, New Share, Pause Share, and Annotate. A green bar indicates 'You are screen sharing' and a red button says 'Stop Share'. The bottom status bar shows 'Page: 16 of 16' and 'Words: 658'. The Windows taskbar is visible at the very bottom with icons for various applications.

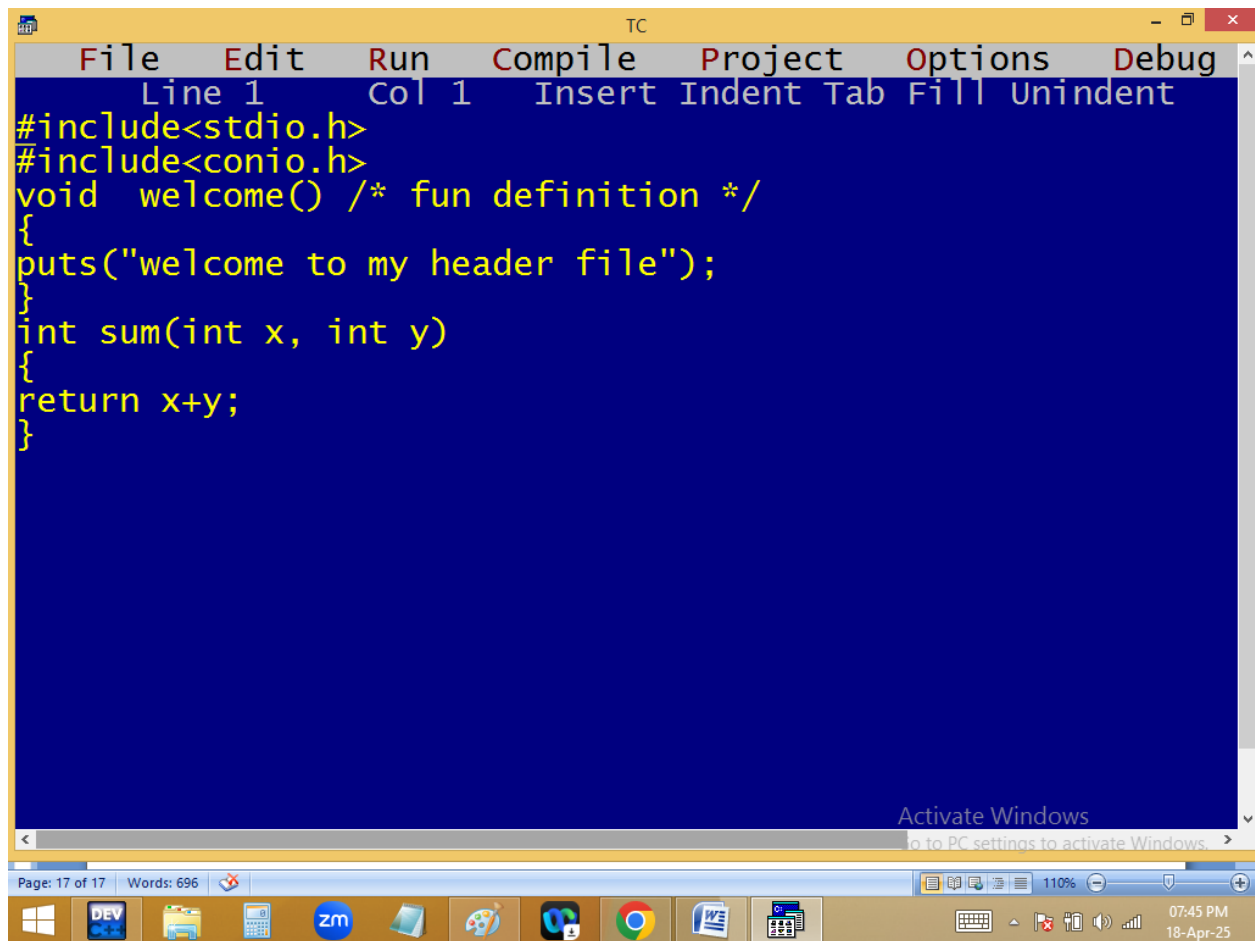
```
File Edit Run C...
Line 7 Col 1 Insert Indent Tab Fill Unindent * E:REV.
#include<stdio.h>
#include<conio.h>
char * max(int *x, int *y)
{
return *x>*y ? "a is big" : *y > *x ? "b is big": "Both are equal";
}
void main()
{
int a,b; char *p;
clrscr();
printf("Enter a, b values "); scanf("%d %d",&a, &b);
p = max(&a, &b);
printf(p); /* fun calling */
getch();
}
```

Page: 16 of 16 Words: 658





**Creating a own header file?**



The screenshot shows the Turbo C++ (TC) IDE interface. The menu bar includes File, Edit, Run, Compile, Project, Options, and Debug. The status bar at the top indicates 'Line 1 Col 1' and provides options for Insert, Indent, Tab, Fill, and Unindent. The main editing area has a dark blue background with yellow text. The code defines two functions: 'welcome' and 'sum'. The 'welcome' function is a void function that prints a message. The 'sum' function is an integer function that returns the sum of two integers. The Windows taskbar at the bottom shows various application icons and the system clock.

```
File Edit Run Compile Project Options Debug
Line 1 Col 1 Insert Indent Tab Fill Unindent
#include<stdio.h>
#include<conio.h>
void welcome() /* fun definition */
{
puts("welcome to my header file");
}
int sum(int x, int y)
{
return x+y;
}
```

Page: 17 of 17 Words: 696 110% 07:45 PM 18-Apr-25

Save as myheader.h [ any name ]

Using header file in a c program?

```
TC
File Edit Run Compile Project Options Debug
Line 1 Col 1 Insert Indent Tab Fill Unindent
#include "myheader.h"
void main()
{
clrscr();
welcome(); /* fun calling */
printf("Sum=%d",sum(10,20));
getch();
}
```

Page: 18 of 18 Words: 703 110% 07:48 PM 18-Apr-25

